```
# ============================================================
# EXPERIMENT 8
# Customer Churn Prediction using Decision Tree Classification
# ============================================================

# -------------------- IMPORT LIBRARIES --------------------
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

# ============================================================
# 1. LOAD DATASET
# ============================================================

print("Upload your Telco Customer Churn CSV file")
uploaded = files.upload()

filename = list(uploaded.keys())[0]
df = pd.read_csv(filename)

print("\nDataset Loaded Successfully!")
print("File:", filename)

# ============================================================
# 2. BASIC DATA EXPLORATION
# ============================================================

print("\n========== FIRST 5 ROWS ==========")
display(df.head())

print("\n========== DATASET SHAPE ==========")
print(df.shape)

print("\n========== COLUMN NAMES ==========")
print(df.columns.tolist())

print("\n========== DATA INFORMATION ==========")
df.info()

print("\n========== MISSING VALUES ==========")
print(df.isnull().sum())

print("\n========== DUPLICATE ROWS ==========")
print(df.duplicated().sum())

# ============================================================
# 3. DATA PREPROCESSING
# ============================================================

# Remove spaces from column names
df.columns = df.columns.str.strip()

# TotalCharges is sometimes stored as text
if "TotalCharges" in df.columns:
    df["TotalCharges"] = pd.to_numeric(
        df["TotalCharges"],
        errors="coerce"
    )

# Remove customer ID because it does not help prediction
if "customerID" in df.columns:
    df = df.drop("customerID", axis=1)

# Remove missing values
df = df.dropna()
```

```
print("\nShape after preprocessing:")
print(df.shape)

# ============================================================
# 4. ENCODE CATEGORICAL VARIABLES
# ============================================================

# Convert target Churn:
# Yes = 1
# No = 0

if "Churn" not in df.columns:
    print("\nERROR: 'Churn' column not found!")
else:

    df["Churn"] = df["Churn"].map({
        "Yes": 1,
        "No": 0
    })

    # Encode all categorical columns
    categorical_columns = df.select_dtypes(
        include=["object"]
    ).columns

    encoder = LabelEncoder()

    for col in categorical_columns:
        df[col] = encoder.fit_transform(df[col])

    print("\nCategorical variables encoded successfully.")

    print("\nProcessed Dataset:")
    display(df.head())

    # ============================================================
    # 5. DEFINE FEATURES AND TARGET
    # ============================================================

    X = df.drop("Churn", axis=1)
    y = df["Churn"]

    print("\nNumber of Features:", X.shape[1])
    print("Target Variable: Churn")

    # ============================================================
    # 6. TRAIN-TEST SPLIT
    # ============================================================

    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.20,
        random_state=42,
        stratify=y
    )

    print("\nTraining Samples:", X_train.shape[0])
    print("Testing Samples:", X_test.shape[0])

    # ============================================================
    # 7. TRAIN DECISION TREE CLASSIFIER
    # ============================================================

    model = DecisionTreeClassifier(
        criterion="gini",
        max_depth=5,
        random_state=42
    )

    model.fit(X_train, y_train)

    print("\nDecision Tree Model Trained Successfully!")

    # ============================================================
    # 8. PREDICTION
    # ============================================================

    y_pred = model.predict(X_test)

    print("\nPredictions Generated Successfully!")

    # ============================================================
    # 9. MODEL EVALUATION
```

```
# 9. MODEL EVALUATION
# ============================================================

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("\n========================================")
print("MODEL PERFORMANCE")
print("========================================")

print("Accuracy  :", round(accuracy, 4))
print("Precision :", round(precision, 4))
print("Recall    :", round(recall, 4))
print("F1-Score  :", round(f1, 4))

# Classification Report
print("\n========== CLASSIFICATION REPORT ==========")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=["No Churn", "Churn"],
        zero_division=0
    )
)

# ============================================================
# 10. CONFUSION MATRIX
# ============================================================

cm = confusion_matrix(y_test, y_pred)

print("\n========== CONFUSION MATRIX ==========")
print(cm)

plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["No Churn", "Churn"],
    yticklabels=["No Churn", "Churn"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Decision Tree")
plt.show()

# ============================================================
# 11. DECISION TREE VISUALIZATION
# ============================================================

plt.figure(figsize=(25, 12))

plot_tree(
    model,
    feature_names=X.columns,
    class_names=["No Churn", "Churn"],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Customer Churn Prediction")
```

```
plt.show()

# ========================================================
# 12. FEATURE IMPORTANCE
# ========================================================

feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

print("\n========== FEATURE IMPORTANCE ==========")
display(feature_importance)

# ========================================================
# FEATURE IMPORTANCE GRAPH
# ========================================================

plt.figure(figsize=(10, 7))

sns.barplot(
    data=feature_importance.head(10),
    x="Importance",
    y="Feature"
)

plt.title("Top 10 Features Influencing Customer Churn")
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.show()

# ========================================================
# 13. CHURN DISTRIBUTION
# ========================================================

plt.figure(figsize=(6, 5))

sns.countplot(
    x=df["Churn"]
)

plt.xticks(
    [0, 1],
    ["No Churn", "Churn"]
)

plt.title("Customer Churn Distribution")
plt.xlabel("Churn Status")
plt.ylabel("Number of Customers")
plt.show()

# ========================================================
# 14. BUSINESS INSIGHTS
# ========================================================

print("\n========================================")
print("BUSINESS INSIGHTS")
print("========================================")

top_features = feature_importance.head(5)["Feature"].tolist()

print("Top factors influencing churn:")
for i, feature in enumerate(top_features, 1):
    print(i, ".", feature)

print("\nPossible Retention Strategies:")
print("1. Identify high-risk customers early.")
print("2. Provide personalized offers to high-risk customers.")
print("3. Improve service quality for customers facing issues.")
print("4. Provide suitable plans and loyalty benefits.")
print("5. Monitor customers with high churn probability.")

# ========================================================
# 15. SAVE RESULT
# ========================================================

result = X_test.copy()
result["Actual_Churn"] = y_test.values
```

```
    result['Predicted_Churn'] = y_pred

    result.to_csv(
        "Decision_Tree_Churn_Predictions.csv",
        index=False
    )

    print("\nResult file saved as:")
    print("Decision_Tree_Churn_Predictions.csv")

    # ============================================================
    # FINAL RESULT
    # ============================================================

    print("\n========================================")
    print("EXPERIMENT COMPLETED SUCCESSFULLY")
    print("========================================")

    print("Accuracy  :", round(accuracy * 100, 2), "%")
    print("Precision :", round(precision * 100, 2), "%")
    print("Recall    :", round(recall * 100, 2), "%")
    print("F1-Score  :", round(f1 * 100, 2), "%")


# ============================================================
# QUESTIONS AND ANSWERS
# ============================================================

# Q1. What is classification, and how does it differ from regression?
# Answer:
# Classification predicts discrete or categorical classes.
# Regression predicts continuous numerical values.
# Example: Churn prediction is classification, while house price
# prediction is regression.

# Q2. Explain the working principle of the Decision Tree algorithm.
# Answer:
# A Decision Tree repeatedly splits the dataset using features
# and conditions. Each split creates branches and the final
# leaf node gives the predicted class.

# Q3. Differentiate between Gini Index and Entropy as splitting criteria.
# Answer:
# Gini Index measures the probability of incorrect classification.
# Entropy measures the impurity or uncertainty in the dataset.
# Both are used to select suitable splits in a Decision Tree.

# Q4. What is a Confusion Matrix? Explain its components.
# Answer:
# A Confusion Matrix compares actual and predicted classes.
#
# TP = True Positive
# TN = True Negative
# FP = False Positive
# FN = False Negative

# Q5. Define Accuracy, Precision, Recall, and F1-Score.
# Answer:
# Accuracy = Correct Predictions / Total Predictions
#
# Precision = TP / (TP + FP)
#
# Recall = TP / (TP + FN)
#
# F1-Score = 2 * (Precision * Recall) /
#            (Precision + Recall)
#
# These metrics help evaluate different aspects of classification
# performance.

# Q6. What is overfitting in a Decision Tree? How can it be reduced?
# Answer:
# Overfitting occurs when a Decision Tree learns the training data
# too closely and performs poorly on unseen data.
# It can be reduced using max_depth, min_samples_split,
# min_samples_leaf, pruning and cross-validation.

# Q7. Why is customer churn prediction important for businesses?
# Answer:
# Churn prediction helps businesses identify customers who may
# leave the service. This allows organizations to take early
# retention actions and reduce customer and revenue loss.

# Q8. What are the advantages and limitations of Decision Tree?
```

```
# Answer:
# Advantages:
# 1. Easy to understand and interpret.
# 2. Requires little data preprocessing.
# 3. Works with numerical and categorical data.
# 4. Useful for explaining decision rules.
#
# Limitations:
# 1. Can overfit easily.
# 2. Small data changes can change the tree.
# 3. Large trees can become complex.

# Q9. Mention three real-world applications of Decision Tree
# Classification other than customer churn prediction.
# Answer:
# 1. Medical disease diagnosis
# 2. Loan approval / credit risk classification
# 3. Spam email detection

# Q10. How can churn prediction insights improve customer retention
# and business profitability?
# Answer:
# Churn prediction identifies important factors related to customer
# leaving. Businesses can target high-risk customers with suitable
# offers, better services, discounts and loyalty programs.
# This can help improve retention and reduce revenue loss.
```